

上海交通大学试卷 (A 卷)

(2020 至 2021 学年 第 2 学期)

班级号_____ 学号_____ 姓名 _____
课程名称_____ 操作系统_____ 成绩 _____

一、 中断与系统调用(28')

1. 现代 CPU 至少有两个特权级，分别用来运行内核和用户程序。为什么要这样设计？ (4')
2. 用户程序执行的过程会触发同步异常或被异步中断打断，此时 CPU 特权级会切换到内核态。在 ChCore 中，用户程序的上下文被保存在内核栈中。这里的内核栈是每个线程独立的还是所有线程共享的？为什么？ (4')
3. Linux 把中断处理过程分为了上半部和下半部。
 - a) 这样设计有什么优点？ (4')
 - b) 考虑一次中断发生后，以下操作哪些应该放在上半部，哪些应该放在下半部？请简要解释原因。 (4')
 - 1) 处理设备通过 DMA 发送的数据
 - 2) 解除中断屏蔽
 - 3) 调用一些可能导致睡眠的函数
 - 4) 读设备寄存器检查设备状态
4. 考虑一个处理网络请求的场景：（左为用户态程序，右为简化的内核态的网卡驱动）

1	// usermode	// kernel
2	...	void do_read() {
3	while (working) {	...
4	read(sockfd, request, len);	if (packet_buffer is empty) {
5	handle(request);	wait_event(queue);
6	}	}
7	...	raw_packet = get(packet_buffer);
8		packet = protocol_stack(raw_packet);
9		copy_to_user(userbuf, packet, len);
10		...
11		}
12		void handle_irq_netdev() {
13		put(packet_buffer, dma_buffer);
14		wake_up(queue);
15		}

面对高速网卡，上述代码难以达到最高性能，此时的性能瓶颈在什么地方？应该如何改进？ (4')

我承诺，我将严格遵守考试纪律。

承诺人：_____

题号	一	二	三	四
得分				
批阅人(流水阅卷教师签名处)				

5. 请分别简要描述读取磁盘时，read()系统调用在宏内核和微内核的执行流程。(4’)
6. Exception 会带来哪些额外的开销？试设计一个 Exception-Less 的系统调用方案，并与传统方案作比较（提示：利用多核）。(4’)

二、 同步(24’)

1. 并行和并发程序中通常需要进行线程间的同步，请举出两个需要同步的例子。(4’)
2. 互斥锁是为了方便并行编程而抽象出来的一种同步原语。Ticket Spinlock、MCS Lock 是常见的互斥锁实现。请简要说明他们各自的特点和适用场景。(4’)
3. 下面的代码是 ticket spinlock 的简单实现，我们发现 lock 结构中的变量都注明了 volatile 关键字，volatile 关键字会告诉编译器不要对相关的变量进行优化。当去掉 volatile 关键字之后，我们发现，有时候本应该轮到某个线程拿到锁，即 lock->owner 已经等于该线程拿到的 ticket，但这个线程仍然一直停留在在第 8 行的循环，无法跳出，导致整个程序无法继续进行。请从编译优化的角度，根据代码具体分析该现象出现的原因，并总结出 volatile 关键字的使用场景。(4’)

```
1 struct lock {
2     volatile int owner;
3     volatile int next;
4 };
5
6 void lock(struct lock *lock) {
7     int my_ticket = atomic_FAA(&lock->next, 1);
8     while(lock->owner != my_ticket) {}
9     barrier();
10 }
11
12 void unlock() {
13     barrier();
14     lock->owner++;
15 }
```

4. 同步原语的使用不当也会导致某些问题。下面代码描述了一个银行账户转账的例子，其使用了互斥锁来保护账户余额的修改。请从同步的角度，分析多个线程执行下面代码时可能会出现什么问题，并提出一种解决方法。(4')

```
1 struct Account {
2     int id;
3     double balance;
4     struct lock *lock;
5 };
6
7
8 int transfer(Account *from, Account *to, double amount) {
9     int err = 0;
10
11     lock(from->lock);
12     lock(to->lock);
13
14     if (from->balance >= amount) {
15         from->balance -= amount;
16         to->balance += amount;
17     } else {
18         err = 1;
19     }
20
21     unlock(to->lock);
22     unlock(from->lock);
23
24     return err;
25 }
```

5. 不同架构的处理器使用不同的内存模型，例如 ARM 架构使用弱一致性模型（弱一致性模型不保证任何对不同的地址的读写操作顺序），Intel x86 架构使用 TSO（Total Store Ordering）模型（TSO 模型保证对不同地址的读-读、读-写、写-写顺序，不保证写-读顺序）。下面的代码是并发哈希表的一种实现，其仅仅用锁保护了 put 操作，我们发现上述实现在某些处理器上总能正确运行，在另一些处理器上，lookup 操作有时会返回错误的值或者找不到已经 put 的值。
- a) 在不考虑编译乱序的情况下，请从内存一致性的角度出发，分析两种处理器架构下出现不同现象的原因。(4')
- b) 请分别用添加锁（lock()和 unlock()）和添加内存屏障（barrier()）的方法修改下述代码，使得修改后的代码能在不同架构的处理器上都能够正确运行，并使得不同的读者可以同时查询同一个 bucket 中的元素。(4')

```

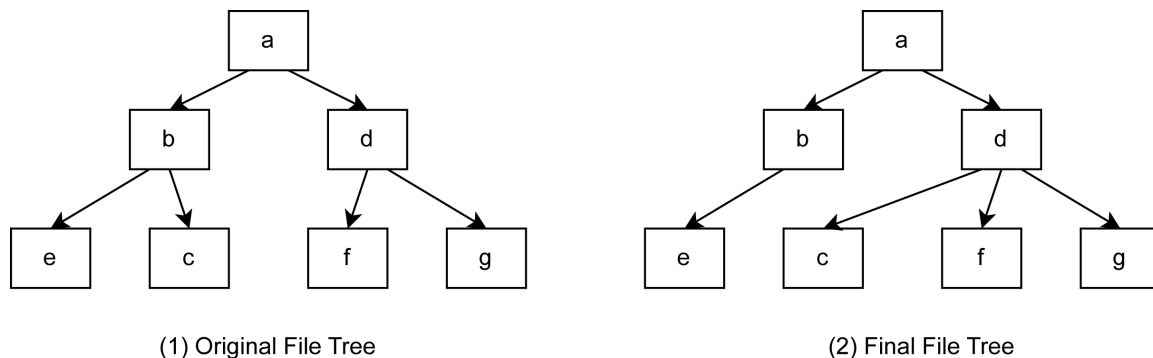
1 struct entry {
2     int key;
3     int value;
4     struct entry *next;
5 };
6
7 struct entry *table[N_BUCKET];
8 struct lock *bucket_locks[N_BUCKET];
9
10 void insert(int key, int value, struct entry **p) {
11     struct entry *e = malloc(sizeof(struct entry));
12     e->key = key;
13     e->value = value;
14     e->next = *p;
15     *p = e;
16 }
17
18 void put(int key, int value) {
19     lock(&bucket_locks[key % N_BUCKET]);
20
21     struct entry **p = &table[key % N_BUCKET];
22     insert(key, value, p);
23
24     unlock(&bucket_locks[key % N_BUCKET]);
25 }
26
27 struct entry* lookup(int key) {
28     struct entry *e = table[key % N_BUCKET];
29     for (; e != NULL; e = e->next) {
30         if(e->key == key) break;
31     }
32     return e;
33 }

```

三、 文件系统(28')

1. 在 Windows 系统下的“本地磁盘(C:)”中有一个非常大的文件其路径为“C:/os/ubuntu_20.04.iso”，当你尝试将其移动到“C:/study/”目录下时，这个操作很快就完成了，而尝试将其移动到“U 盘(E:)”下时却需要花费很多时间，请解释发生这种现象的原因。(4')
2. Ordered Mode 和 Journaling Mode 分别是 Ext4 文件系统的两个运行模式。
 - a) 请简述这两种模式的流程。(4')
 - b) 请从性能和崩溃一致性两个方面说明两种模式的区别。(4')

3. Copy-on-write 是一种在 NVM 文件系统上常用的保证操作原子性的方法，请描述 Copy-on-write 原子性地完成 rename /a/b/c /a/d/c 操作的过程，将下图目录树(1)转化为(2)。(4')



4. 请阅读如下代码，并回答问题：

```

1  int main() {
2      int fd;
3      char *addr;
4      char buf[256];
5
6      memset(buf, 0, 256);
7
8      fd = open("./b.txt", O_RDWR);
9      addr = mmap(NULL, 10, PROT_WRITE, MAP_SHARED, fd, 0);
10     printf("Before Assign:\t%s\n", (char *)addr);
11     addr[5] = 'Y';
12     printf("After Assign:\t%s\n", (char *)addr);
13     read(fd, buf, 10);
14     printf("Read From fd:\t%s\n", (char *)buf);
15
16     close(fd);
17 }

```

输出：

```

1  Before Assign: 1234567890
2  After Assign:  12345Y7890
3  Read From fd:  12345Y7890

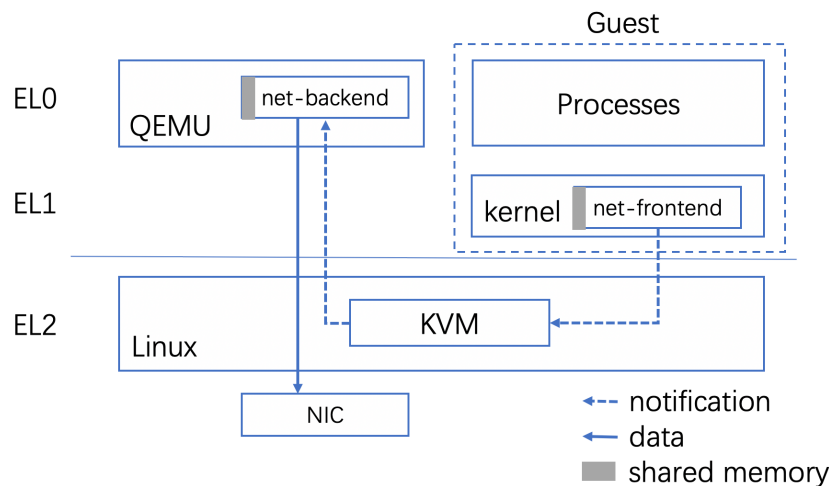
```

- a) 小雪在 ubuntu 20.04 中用 c 语言编写了如上代码并获得了如上的输出，她发现使用 mmap 将文件映射成内存后修改，再立刻使用传统的 read 接口读取 fd 中的数据，仍然可以获得修改后的最新数据。而她马上想到只是修改虚拟内存中的数据并不会触发系统调用写磁盘文件，你能帮她解释为什么 read 函数仍然可以读到最新的数据吗？(4')

- b) 小雪随后在 ubuntu 20.04 中, 尝试读取一个非常大的文件, 她发现用 read 系统调用把文件读入内存需要很长的时间, 但换一种方法, 调用 mmap 时函数立刻就返回, 并且随后马上可以在内存正常读写数据了, 造成这种现象的原因是什么? (4')
- c) 小雪尝试在 ChCore 微内核中实现 mmap 映射文件的功能, 她查阅资料发现, 宏内核中文件系统作为内核模块在内核态工作, 而微内核中文件系统作为一个用户程序运行在系统中。显然, 宏内核的 mmap 设计在微内核中并不适用, 请结合微内核设计的特点, 列出一条微内核中实现 mmap 会遇到的问题并解释原因。(4')

四、 虚拟化(20')

- DMA: 是一种快速传送数据的机制。DMA 传输将数据从一个地址空间复制到另外一个地址空间。CPU 负责初始化这个传输动作, 而传输动作本身是由 DMA 控制器来实行和完成。最初的 DMA 地址是物理地址, 但这会带来安全问题, 因此出现了 IOMMU 硬件。
 - 使用 IOMMU 之后, 设备看到的还是物理地址吗? 设备访问内存的过程是怎样的? (4')
 - 在使用 IOMMU 之前, 设备访问主机内存会有什么安全问题? IOMMU 是如何解决它的? 除了能够解决安全问题, IOMMU 提供的抽象还有什么别的好处? (4')
- 虚拟机监控器分为 Type-1 和 Type-2 两种, 它们的设计各有什么优点? 你能各举出两种类型虚拟机监控器的一个例子吗? (4')
- IO 虚拟化



- 上图展示了 guest 发送网络包时, data 和 notification 的传输路径(guest 和 qemu 通过 share memory 共享数据), 这种设计发送网络包的延迟很高, 你认为影响它性能的原因有哪些? (至少提出两个) (4')
- 为了解决 3.a 的性能问题, 有人提出可以将 net-backend 放在 host kernel 中(backend 和 frontend 仍通过 share memory 共享数据), 它从哪两个方面解决了 3.a 的性能问题? (4')